

Best of Both Worlds: AutoML Codesign of a CNN and its Hardware Accelerator

Mohamed S. Abdelfattah¹, Łukasz Dudziak¹, Thomas Chau¹, Royson Lee¹, Hyeji Kim¹, Nicholas D. Lane^{1,2}

¹*Samsung AI Center - Cambridge*, ²*University of Oxford*
mohamed1.a@samsung.com

Abstract—Neural architecture search (NAS) has been very successful at outperforming human-designed convolutional neural networks (CNN) in accuracy, and when hardware information is present, latency as well. However, NAS-designed CNNs typically have a complicated topology, therefore, it may be difficult to design a custom hardware (HW) accelerator for such CNNs. We automate HW-CNN codesign using NAS by including parameters from both the CNN model and the HW accelerator, and we jointly search for the best model-accelerator pair that boosts accuracy and efficiency. We call this Codesign-NAS. In this paper we focus on defining the Codesign-NAS multiobjective optimization problem, demonstrating its effectiveness, and exploring different ways of navigating the codesign search space. For CIFAR-10 image classification, we enumerate close to 4 billion model-accelerator pairs, and find the Pareto frontier within that large search space. This allows us to evaluate three different reinforcement-learning-based search strategies. Finally, compared to ResNet on its most optimal HW accelerator from within our HW design space, we improve on CIFAR-100 classification accuracy by 1.3% while simultaneously increasing performance/area by 41% in just ~1000 GPU-hours of running Codesign-NAS.

I. INTRODUCTION

In the evermore important field of machine learning, there are two trends that are gaining importance and increasing in both volume of academic research and number of industrial deployment. First, the design of deep neural networks (DNNs) is becoming automated through the general field of automated machine learning (AutoML) and more specifically neural architecture search (NAS). Instead of manually designing a DNN, NAS automatically searches through hundreds or thousands of models using reinforcement learning (RL), evolutionary algorithms or other approaches, to hone in on the best DNN model. The second trend is pertaining to the hardware (HW) that runs DNN algorithms. Instead of using commodity HW, there is an increasing number of custom accelerators, either based on FPGAs [1], [2] or ASICs [3], [4]. In this work, we combine NAS for DNN model discovery and HW design space exploration (DSE) to automatically codesign both the DNN model and its HW accelerator. We expose parameters from both the DNN and the HW thus allowing the bottom-up design of the two parts taking into account accuracy and efficiency of the overall design. This, in turn, allows us to tailor the DNN to the HW and vice versa.

Related Work. NAS has been successful in discovering DNN models that achieve state-of-the-art accuracy on image classification [5], super-resolution [6], speech recognition [7] and machine translation [8]. HW-aware NAS adds latency to the reward function so that discovered models optimize both accuracy and inference latency, for example, when running

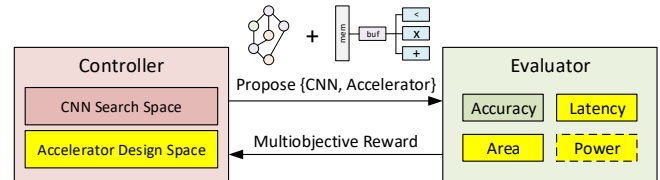


Fig. 1: Codesign-NAS system with differences to conventional NAS systems highlighted in yellow.

on mobile devices [9]. More recently, reinforcement learning-based *codesign* has been introduced to automate both the discovery of a DNN model and its partitioning across multiple FPGAs based on a theoretical utilization model for each device [10]. Additionally, the authors in [11] propose an automatic codesign methodology based on stochastic coordinate descent to refine a DNN search space based on hardware constraints, then codesign the DNN using the refined DNN operations and a parameterizable hardware accelerator.

Contributions. We use reinforcement learning to automatically codesign CNN and HW architecture, investigate different RL-based search strategies to navigate the codesign search space and demonstrate the efficacy of our search strategies under different scenarios, and finally, we use Codesign-NAS to simultaneously improve accuracy and efficiency of image classification on a popular FPGA platform. Fig 1 illustrates our system: Codesign-NAS. A controller selects a CNN architecture from a CNN search space and a HW architecture from an accelerator design space. Both are sent to the evaluator that implements the CNN on the proposed accelerator to find accuracy and efficiency metrics, such as latency, area and power. All metrics are then used to create a multiobjective reward that influences the controller to find better CNN-HW pairs. Our contributions are:

- 1) Propose a general formulation for Codesign-NAS to automatically search for CNN-HW pairs using RL and optimize multiple objectives (area, latency, accuracy).
- 2) Enumerate ~4 billion model-accelerator pairs to study the Pareto-front in a representative codesign search space, and propose three different search strategies to navigate the codesign search space.
- 3) Demonstrate the effectiveness of Codesign-NAS by using it for the task of CIFAR-100 image classification. We find CNN-HW pairs that outperform both ResNet [12] and GoogLeNet [13] even when paired with their most-optimal HW accelerators.

II. APPROACH

In this section we outline our Codesign-NAS system with a focus on the CNN-HW search spaces, the multiobjective optimization (MOO) problem, and the accelerator HW model.

A. Codesign Multiobjective Neural Architecture Search

NAS focuses on searching for the best parametrization of a predefined model architecture by making a number of *decisions* and evaluating performance of the model when constructed according to the chosen options. We can define our codesign optimization problem addressed by NAS as:

$$S = O_{nm1} \times O_{nm2} \times \dots \times O_{hw1} \times O_{hw2} \times \dots$$

$$s^* = \underset{s \in S}{\operatorname{argmax}} \mathcal{E}(s) \quad (1)$$

where O_{nm} is a DNN option, such as the selection of an operation, and O_{hw} is a hardware option such as the size of a buffer. $\mathcal{E}$ is the evaluation function to find the performance of a search point s ; that is, finding the accuracy and efficiency metrics of running the discovered DNN on the discovered HW.

Since enumerating all points from S is often infeasible in practice due to their large number and time-consuming evaluation, the main challenge of NAS is to produce as good an approximation of s^* as possible while being allowed to evaluate only a limited number of architectures. Therefore, for a sequence of T explored architectures (search steps): $\tau(T) = (s_1, s_2, \dots, s_T)$, Eq. 1 takes the following form:

$$s^* \approx s^* = \underset{s \in \tau(T)}{\operatorname{argmax}} \mathcal{E}(s) \quad (2)$$

and the main focus is to guarantee that the search is able to explore points which optimize $\mathcal{E}$ as T increases.

In this work, we use a probabilistic, trainable policy π_θ to guide the search as proposed in prior work [5]. At each search step t the policy (implemented as a single LSTM cell followed by a linear layer as in [5]) is first sampled in order to get a structure sequence s_t and later updated using REINFORCE and stochastic gradient descent: $\nabla_{\theta} \pi_{\theta}(s_t) \mathcal{E}(s_t)$.

We consider the following three *quality metrics* when assessing a DNN-accelerator pair: accuracy of the DNN, area of the accelerator and latency of the DNN when run on the accelerator. In order to link the three objectives to the original optimization problem from Eq. 1, we consider a mixture of two standard approaches: we first limit the set of points in S by providing a set of thresholds for all/some of the metrics and filter out points with at least one of them being above/below the threshold (ϵ -constraint [14]); we then take the weighted sum of the remaining ones [14]. This is expressed by the following *reward* function $\mathcal{R}$:

$$\mathcal{R} : \{\mathbf{m} \mid \mathbf{m} \in \mathbb{R}^n \wedge \forall_i [\mathbf{m}_i \geq \mathbf{th}_i]\} \rightarrow \mathbb{R}$$

$$\mathcal{R}(\mathbf{m}) = \mathbf{w} \cdot \mathcal{N}(\mathbf{m}) \quad (3)$$

where $\mathcal{N}$ is a linear element-wise normalization function which maps values from the range $(x_{\min}, x_{\max})$ to $(0, 1)$, $\mathbf{m}$ is a vector of metrics, $\mathbf{th}$ is a vector of thresholds and $\mathbf{w}$ is a vector of weights. The evaluation function, which is the subject of optimization according to Eq. 1, is defined as:

$$\mathcal{E}(s) = \mathcal{R}(-\text{area}(s), -\text{lat}(s), \text{acc}(s)) \quad (4)$$

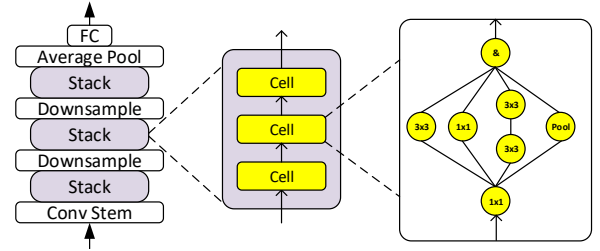


Fig. 2: NASBench [15] CNN architecture.

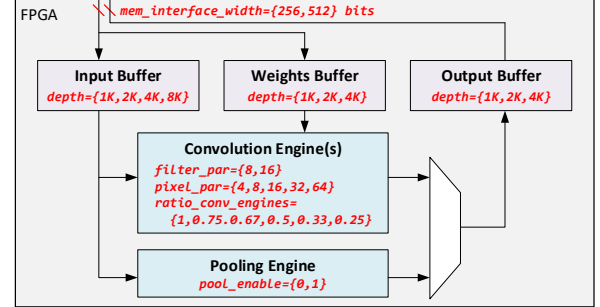


Fig. 3: CHaiDNN [16] FPGA accelerator with configurable parameters highlighted in red.

If a search point does not meet specified constraints, a *punishment* function $\mathcal{R}_v$ (with opposite sign to the reward) is used as feedback for the RL controller to deter it from searching for similarly *bad* points.

B. Codesign Search Space

1) *CNN Search Space*: NASBench [15] provides a CNN search space for image classification including evaluation statistics such as accuracy and training time on the CIFAR-10 dataset for ~423 thousand unique CNNs. Fig. 2 shows the structure of the CNNs within NASBench. The only varying part of each model is the inner-most design of a single cell, which is then repeated three times to form a stack. At most 7 operations and 9 connections are allowed per cell, and addition/concatenation operations are inserted automatically according to a set of rules – for more information, please refer to NASBench [15]. In Section III, we use the NASBench database of precomputed accuracy to find exactly the Pareto-optimal points within the codesign space (Equation 1), we then compare them to the points found by our search (Equation 2) under different search strategies and optimization targets.

2) *Accelerator Design Space*: We base our work on CHaiDNN – a library for acceleration of CNNs on System-on-chip FPGAs [16]. The FPGA accelerator supports convolution and pooling operations while the unsupported layers run on the CPU. The accelerator is configurable to maximize the hardware efficiency on different CNNs. Fig. 3 shows the accelerator parameters and their valid values. The available parameters are fairly standard ones in custom hardware accelerators, configuring things like buffer depths, external memory interface width, and the amount of parallelism in the filter and pixel dimensions. We add the parameter *ratio_conv_engines* to CHaiDNN. In its default configuration, CHaiDNN sets *ratio_conv_engines* = 1, which means that a single gen-

TABLE I: Estimated FPGA block area for Zync Ultrascale+.

Resource	Relative Area (CLB)	Tile Area (mm^2)
CLB	1	0.0044
BRAM - 36 Kbit	6	0.026
DSP	10	0.044
Total	64,922	286

eral convolution engine runs any type of convolution. When *ratio_conv_engines* is set to any number below 1, there are two convolution engines - one of them specialized for 3x3 filters, and the other for 1x1 filters – the ratio determines the number of DSPs assigned to each convolution engine. These parameters form 8640 different combinations of valid CHaiDNN accelerators, and is representative of a relatively simple hardware design space. While we use this search space to implement our system and prove its effectiveness, we believe that much more parameter-rich hardware design space can be equally leveraged using our methodology, and would be expected to achieve larger improvements.

C. Accelerator Modelling

To avoid time-consuming compilations/runs on the FPGA, we create area/latency models to use with Codesign-NAS.

1) *Area model*: We divide the accelerator into its different components: convolution engine, buffers, pooling engine, memory interface, and we create area models based on the utilization of configurable logic blocks (CLB), digital signal processors (DSP) and block RAM (BRAM). For each component, we break it down further into simpler subcomponents – for example, a sliding window buffer within the convolution engine that is parameterized with *pixel_par* and *filter_par* would be modeled with an equation that takes these 2 variables as input. We verified our area model against 10 full FPGA compilations with different parameters and our model had on average 1.6% error – we felt this was adequate for area estimation. Based on the FPGA resource utilization, we estimate the accelerator size in mm^2 such that area is quantified by a single number – silicon area¹ according to Table I.

2) *Latency model*: The latency model consists of two parts: 1) latency lookup table of operations and 2) scheduler. In our CNN search space, there are 85 unique variations of convolutions, pooling and element-wise operations (different input/filter sizes etc.). We run each operation on the FPGA accelerator with different parameters and measure latency which we then store in a lookup table. The scheduler assigns operations to the parallel compute units greedily and calculates the total latency of the CNN model using the lookup table. To validate the latency model, we pick the CNN model from NASBench with the GoogLeNet cell, and we run it on 10 different accelerator variants with different parameterizations. Our latency model is 85% accurate on this validation set - there is room for improving our per-layer latency model, but we leave that to future work.

¹Silicon area is not available for Zync Ultrascale+ devices so we use area for similar devices [17] and account for the process node (20nm vs. 40nm) and the different block properties (8 LUTs per CLB instead of 10, and 36 Kbit per BRAM instead of 9 Kbit).

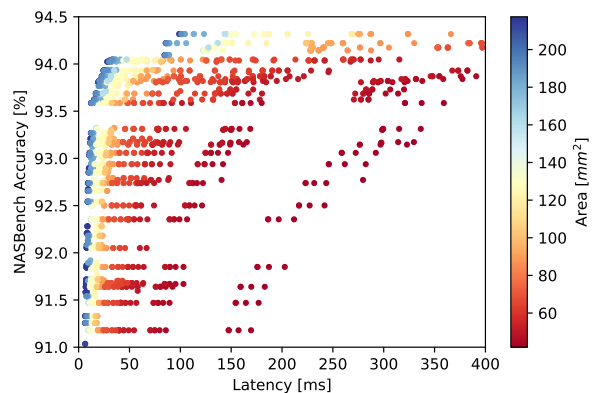


Fig. 4: Pareto-optimal points in the codesign search space.

III. SEARCH INVESTIGATION ON NASBENCH-101

In this section, we analyze the codesign search space using the NASBench dataset and our accelerator latency/area model. The CNN accuracy in NASBench is precomputed and stored in a database, and our FPGA accelerator model runs quickly on a desktop computer. This allows us to enumerate the entire search space, consisting of 3.7 billion data points, and find the Pareto-optimal points within that space. Finally, we investigate how to automatically navigate the codesign search space using our RL-based methodology described in Section II. In that context, we evaluate three search strategies in terms of proximity of discovered points to the Pareto-optimal ones and the search convergence speed.

A. Pareto-Optimal Points

To understand the *good* points in the codesign search space, we look for Pareto-optimal [14] points within the 3.7 billion model-accelerator pairs. This is done iteratively by filtering dominated points from the search space. The remaining (non-dominated) points are better in at least one of our evaluation metrics (area, latency or accuracy) w.r.t. any other point. For our search space, there were only 3096 Pareto-optimal CNN-HW pairs – these are illustrated in Fig. 4.

As Fig 4 shows, there is a three-way tradeoff between area, latency and accuracy – to improve a metric, one or both of the other two must degrade. The Pareto-optimal points form concentric accuracy-latency tradeoff curves, each at a different accelerator area – different points on the y-axis represent different CNNs, and different points on the x-axis are different accelerator designs. Fig. 4 highlights the diversity and number of *good* points, and motivates why automated codesign techniques are necessary. First, less than 0.0001% of points in the search space were actually Pareto-optimal – it is near impossible to manually pin-point these model-accelerator pairs. Second, the Pareto-optimal points are very diverse and include 338 accelerator variants and 136 different CNN cells – it is very difficult to manually translate from accuracy/efficiency requirements to an optimal point. In the next two subsections we present and evaluate NAS search strategies that find codesign points close to these Pareto-optimal solutions.

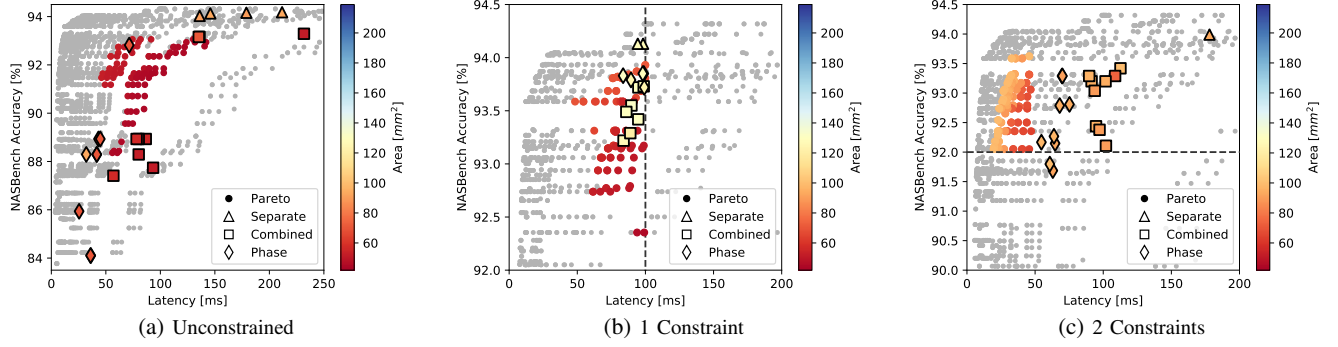


Fig. 5: Top search results compared to the top 100 Pareto-optimal points that maximize each experiment's reward function.

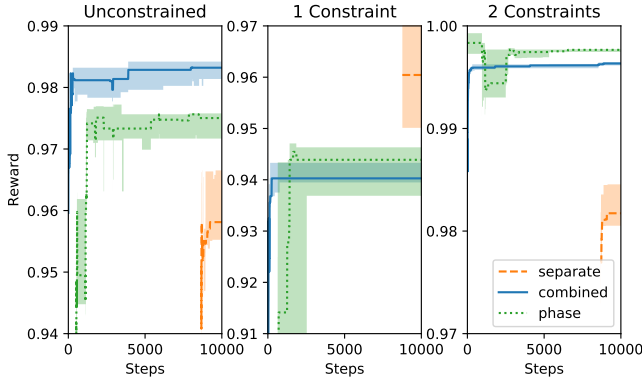


Fig. 6: Reward values of separate, combined and phase strategies for our 3 search scenarios. Note that separate search only has a MOO objective in its second part when it is optimizing the HW accelerator for the discovered CNN.

B. Search Strategies

1) *Combined Search*: The first search strategy is to consider both sub-search spaces together as in Equation 1 and apply REINFORCE directly – we call this *combined* search. This strategy has the ability to update both the CNN and the accelerator in each step, and is therefore able to make faster changes to adapt to the reward function. However, the *combined* search space is much larger, which may make it more difficult to find the best points. In this approach we run each experiment for 10,000 steps.

2) *Phase Search*: We explicitly specify specialized phases during the search by freezing one part of the search space (e.g. a specific accelerator) and only focus on the other (e.g. a CNN design). We would then select the best found CNN, and switch to the accelerator phase to search for suitable hardware. The two phases are interleaved and repeated multiple times in order to find a globally optimal solution. This requires us to have two different controllers – one which only learns to select the best combination of options for the FPGA design and the other one to optimize the CNN structure. This divide-and-conquer technique may make it easier to find better locally-optimal points (per search space). However, mutual impact between the phases is limited, which may make it more difficult to adapt the CNN and the accelerator to each other optimally. When running phase search, we set number of steps for each CNN

phase to 1000, and 200 steps for each HW phase, repeating them until we hit the total of 10,000 steps.

3) *Separate Search (baseline)*: We compare our proposed codesign search strategies above to a baseline where we separately search for a CNN, followed by design-space exploration of accelerator parameters. This methodology is similar to the conventional, sequential design of the two parts. We run the *separate* experiments for the total of 10,000 steps splitting the two phases into 8,333 and 1,667 steps respectively.

C. Search Results

We evaluate our search strategies using three experiments:

- 1) **Unconstrained**: Zero constraints and we arbitrarily² choose the MOO weights as follows $w(area, lat, acc) = (0.1, 0.8, 0.1)$. This scenario may be useful to simply search for many *good* points to understand the codesign search space.
- 2) **1 Constraint**: One constraint on latency $lat < 100ms$ and $w(area, lat, acc) = (0.1, 0, 0.9)$. This scenario is similar to when an end-user may know the task and real-time requirements, but is not sure which FPGA device to choose – the best accuracy at each device size may aid such decision.
- 3) **2 Constraints**: Two constraints on area and accuracy $acc > 0.92, area < 100mm^2$ and we optimize latency (single objective). This occurs when there is a maximum FPGA area budget and a minimum tolerated accuracy for an application – this is a common use-case.

Fig. 5 plots the top 100 Pareto-optimal points that maximize each experiment's reward. Therefore, a good search algorithm would produce results in the vicinity of these top Pareto-optimal points. We also plot the top result from our 3 search strategies, and we repeat each experiment 10 times. Therefore, we have a maximum of 10 points per search strategy for each plot. Fig. 6 shows the reward function (averaged over 10 experiments) for each experiment. Note that we only plot the reward function $\mathcal{R}$ and we do not display the punishment function $\mathcal{R}_v$ on the plot. In summary, the results show the following trends.

²The choice of weights is critical in determining the neighbourhood of good points explored, but we do not study that in this work. We refer the reader to prior literature for an in-depth analysis [14] and a relevant case study [9].

1) *Separate Search*: “Separate” search cannot consistently find good points within the constraints. This is because it searches for the most accurate CNN model without any context of the HW target platform. Fig 5b shows two *lucky* “separate” points that are superior to other searches (and that is also reflected by a higher reward in Fig. 6). However, the plots do not show that the 8 remaining points all have latencies that are much higher than the constraint. This is true for all plots in Fig. 5, where only a few “separate” points fit within the displayed axes, while the rest of the points are generally high accuracy but very low efficiency. This shows the *randomness* of CNNs that are designed without HW context – they may or may not fall within efficiency constraints based on chance, even if the accelerator is heavily tuned for the separately-found CNN, further motivating the need of joint codesign.

2) *Combined and Phase Searches*: These two search strategies improve upon separate search as they take the HW accelerator into account, and more importantly, they consider all variants of the hardware accelerator and all variants of the CNN simultaneously. Fig. 6 shows that “combined” is generally better in the Unconstrained experiment, whereas “phase” achieves a higher reward with both the constrained experiments. This is also highlighted in Fig. 5c that clearly shows that phase gets closer to the ideal points. However, the same figure shows a shortcoming of “phase” search. It is more prone to missing the specified constraints, likely because there are only limited opportunities to switch from the CNN phase to the FPGA phase within the 10,000 steps in our experiment – if we increase the number of search steps, we expect these two experiments to also find points within the constraints. More generally, we can say that phase search is slower to converge, compared to combined search. This is also highlighted in Fig. 6 where phase search goes through a few exploration phases before finding its best result. In summary, we believe that both of these search techniques have their merits; combined works better when the search is unconstrained and is generally faster to converge to a solution, while phase finds better points when there are constraints but typically requires more search steps to do so.

IV. CIFAR-100 CNN-ACCELERATOR CODESIGN

In this section we use Codesign-NAS to discover a CNN model-accelerator pair that optimizes the task of CIFAR-100 image classification. We show that Codesign-NAS can exceed both the efficiency and accuracy of well-known CNN architectures even when paired with their optimal accelerators.

A. Experimental Setup

Unlike our use of NASBench in previous sections, we have no precomputed results for CIFAR-100 image classification, so we must train all discovered CNNs from scratch. However, we still use the same codesign search space as defined in Section II to be able to easily reuse our reliable FPGA latency and area models which were verified on our CNN search space. We use the same training parameters shown in previous work [15] with 108 epochs of training, standard data augmentation (padding, random crop and flipping), initial learning rate of 0.1 with

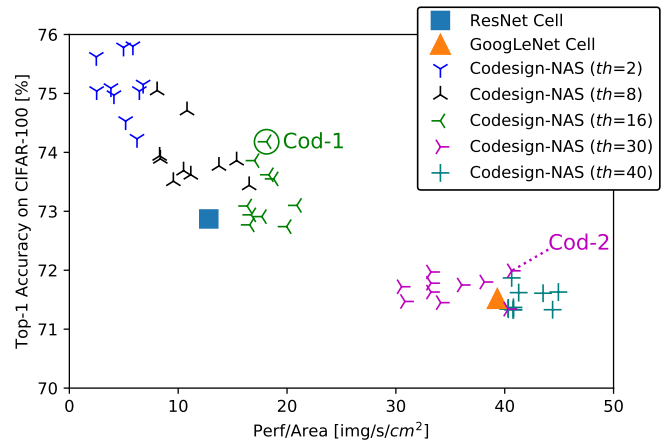


Fig. 7: Top 10 points discovered by Codesign-NAS at each threshold value, compared to ResNet and GoogLeNet cells.

TABLE II: Best points found by Codesign-NAS compared to ResNet and GoogLeNet cells on their best accelerators.

CNN	Accuracy [%]	Perf/Area [img/s/cm ²]	Latency [ms]	Area [mm ²]
ResNet Cell	72.9	12.8	42.0	186
Cod-1	74.2 (+1.3%)	18.1 (+41%)	41.8 (-0.5%)	132 (-29%)
GoogLeNet Cell	71.5	39.3	19.3	132 (-0.8%)
Cod-2	72.0 (+0.5%)	40.6 (+3.3%)	18.5 (-4.2%)	133

cosine decay and weights decay of 10^{-4} . Training every new CNN takes approximately 1-GPU hour, so to be able to train many models, we parallelize Codesign-NAS over 6 machines, each with 8 Nvidia-1080 GPUs each, allowing us to train 48 models in parallel.

We run Codesign-NAS with two constraints combined into one. Specifically, we combine latency and area into performance-per-area ($perf/area$) and constrain the $perf/area$ to a threshold value. We then attempt to maximize accuracy under the constraint. For our RL controller, we gradually increase the $perf/area$ threshold according to (2, 8, 16, 30, 40) – we run the search for ~2300 valid points in total, starting with 300 points at the first threshold value and increasing to 1000 points at the last threshold value. We found that this gradual increase in threshold makes it easier for the RL controller to learn the structure of high-accuracy CNNs. We decided to use the “combined” search strategy described in Section III-B1 as it has shown to be faster to converge.

B. Results

Fig. 7 shows the top-1 accuracy and $perf/area$ of various points searched by Codesign-NAS. We plot the top 10 points among the model-accelerator pairs visited at each threshold value. The plot also shows the ResNet and GoogLeNet cells within our CNN skeleton³ (Fig. 2), and we pair those with their most optimal accelerator in terms of $perf/area$. This is a difficult baseline to beat as we are comparing against two well-known high-accuracy CNN cells when implemented on their

³We believe that it is fair to compare discovered cells from Codesign-NAS GoogLeNet and ResNet cells within the same skeleton (in Fig. 2). Alternatively, we can also insert our discovered cells within the ResNet-50 [12] or GoogLeNet v1 [13] skeletons, and we anticipate similar findings.

TABLE III: HW of best points found by Codesign-NAS.

HW Parameter	Cod-1	Cod-2
filter_par, pixel_par	(16, 64)	(16, 64)
buffer_depths	(4K, 2K, 4K)	(8K, 2K, 2K)
mem_interface_width	256	512
pool_en	false	false
ratio_conv_engines	0.33	0.25

best possible corresponding accelerator in our FPGA search space. However, as the plot shows, we find many points that exceed both the accuracy and efficiency of both the ResNet and GoogLeNet baselines.

We highlight the best two among those points, and label them “Cod-1” and “Cod-2” on Fig. 7. Cod-1 improves upon ResNet by 1.3% accuracy while simultaneously improving *perf/area* by 41% – considerable gains on both accuracy and efficiency. Cod-2 shows more modest improvements over GoogLeNet as shown in Table II, but still beats it on both efficiency and accuracy while running 4.2% faster in terms of absolute latency.

Fig. 8 shows the CNN cell structure of Cod-1 and Cod-2, and Table III lists the HW parameters. It is difficult to reason about automatically designed CNNs [5], but we will highlight observations of our codesigned model-accelerator pairs. For example, the Cod-1 CNN manages to beat ResNet accuracy but uses an important ResNet feature: skip connections and element-wise addition as shown by the rightmost branch of the cell in Fig. 8a. On the hardware side, both Cod-1 and Cod-2 use the largest convolution engine and avoid the use of a dedicated pooling engine. However, the other HW parameters are tailored for each CNN. For example, both the input buffer size and the memory interface width are smaller for Cod-1, likely due to the fact that the Cod-1 CNN uses a larger number of smaller convolutions compared to Cod-2.

Naturally, we anticipate that there might even be better points within our search space that has ~3.7 billion points in total. We only explore ~1000 points before finding Cod-1 and ~2000 points before finding Cod-2 (compared to 8000 points in prior work to discover a CNN [9]). This highlights the speed of convergence of our RL controller and its effectiveness in finding good designs, especially when properly tuned with representative reward functions and search strategies as we have shown in this paper.

V. CONCLUSION

We proposed the automatic codesign of CNN and HW accelerator, and provided a full methodology and case study using FPGAs to support our proposal. We presented three search strategies based on reinforcement learning and compared them against each other and against Pareto-optimal designs. Finally, we implemented Codesign-NAS for the task of CIFAR-100 image classification on a popular FPGA platform and showed that we can improve upon ResNet (paired with its ideal accelerator) by 1.3% in accuracy and 41% efficiency. These are large improvements, especially considering that our FPGA search space contains a limited set of configurable HW parameters. We believe that our findings provide a compelling case for the automated codesign of HW and DNNs. In the

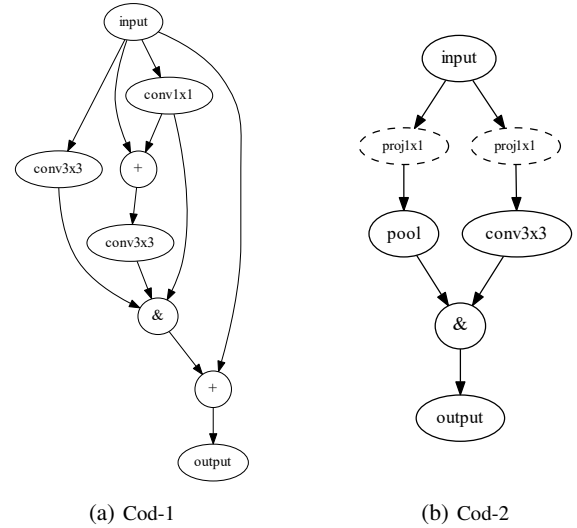


Fig. 8: Best CNN cells discovered by Codesign-NAS that improve upon (a) ResNet and (b) GoogLeNet cells.

future, we hope to study more interesting HW search spaces that give more freedom to Codesign-NAS to tailor a hardware platform for a codesigned DNN.

REFERENCES

- [1] J. Fowers, K. Ovtcharov, M. Papamichael *et al.*, “A Configurable Cloud-scale DNN Processor for Real-time AI,” in *International Symposium on Computer Architecture (ISCA)*.
- [2] M. S. Abdelfattah, D. Han, A. Bitar *et al.*, “DLA: Compiler and FPGA Overlay for Neural Network Inference Acceleration,” in *International Conference on Field Programmable Logic and Applications (FPL)*.
- [3] Y. Chen, T. Yang, J. Emer *et al.*, “Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices,” *Emerging and Selected Topics in Circuits and Systems*, vol. 9, no. 2, June 2019.
- [4] N. P. Jouppi, C. Young, N. Patil *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *International Symposium on Computer Architecture (ISCA)*.
- [5] B. Zoph and Q. V. Le, “Neural Architecture Search with Reinforcement Learning,” *arXiv e-prints*, Nov 2016.
- [6] X. Chu, B. Zhang, H. Ma *et al.*, “Fast, Accurate and Lightweight Super-Resolution with Neural Architecture Search,” *arXiv e-prints*, Jan 2019.
- [7] L. Dudziak, M. S. Abdelfattah, R. Vipperla *et al.*, “ShrinkML: End-to-End ASR Model Compression Using Reinforcement Learning,” in *Proc. Interspeech 2019*, pp. 2235–2239. [Online]. Available: <http://dx.doi.org/10.21437/Interspeech.2019-2811>
- [8] D. R. So, C. Liang, and Q. V. Le, “The Evolved Transformer,” *arXiv e-prints*, Jan 2019.
- [9] M. Tan, B. Chen, R. Pang *et al.*, “MnasNet: Platform-Aware Neural Architecture Search for Mobile,” *arXiv e-prints*, Jul 2018.
- [10] W. Jiang, L. Yang, E. Sha *et al.*, “Hardware/Software Co-Exploration of Neural Architectures,” *arXiv e-prints*, Jul 2019.
- [11] C. Hao, X. Zhang, Y. Li *et al.*, “Fpga/dnn co-design: An efficient design methodology for iot intelligence on the edge,” in *Design Automation Conference (DAC)*.
- [12] K. He, X. Zhang, S. Ren *et al.*, “Deep Residual Learning for Image Recognition,” *arXiv e-prints*, Dec 2015.
- [13] C. Szegedy, W. Liu, Y. Jia *et al.*, “Going Deeper with Convolutions,” *arXiv e-prints*, Sep 2014.
- [14] J. Branke, K. Deb, K. Miettinen *et al.*, *Multiobjective Optimization, Interactive and Evolutionary Approaches*, Springer.
- [15] C. Ying, A. Klein, E. Real *et al.*, “NAS-Bench-101: Towards Reproducible Neural Architecture Search,” *arXiv e-prints*, Feb 2019.
- [16] X. Inc., “Chaidnnv2 - hls based dnn accelerator library for xilinx ultrascale+ mpsoes,” <https://github.com/Xilinx/CHaiDNN>, 2019.
- [17] M. S. Abdelfattah and V. Betz, “Design Tradeoffs for Hard and Soft FPGA-based Networks-on-Chip,” in *International Conference on Field Programmable Technology (FPT)*, pp. 95–103.